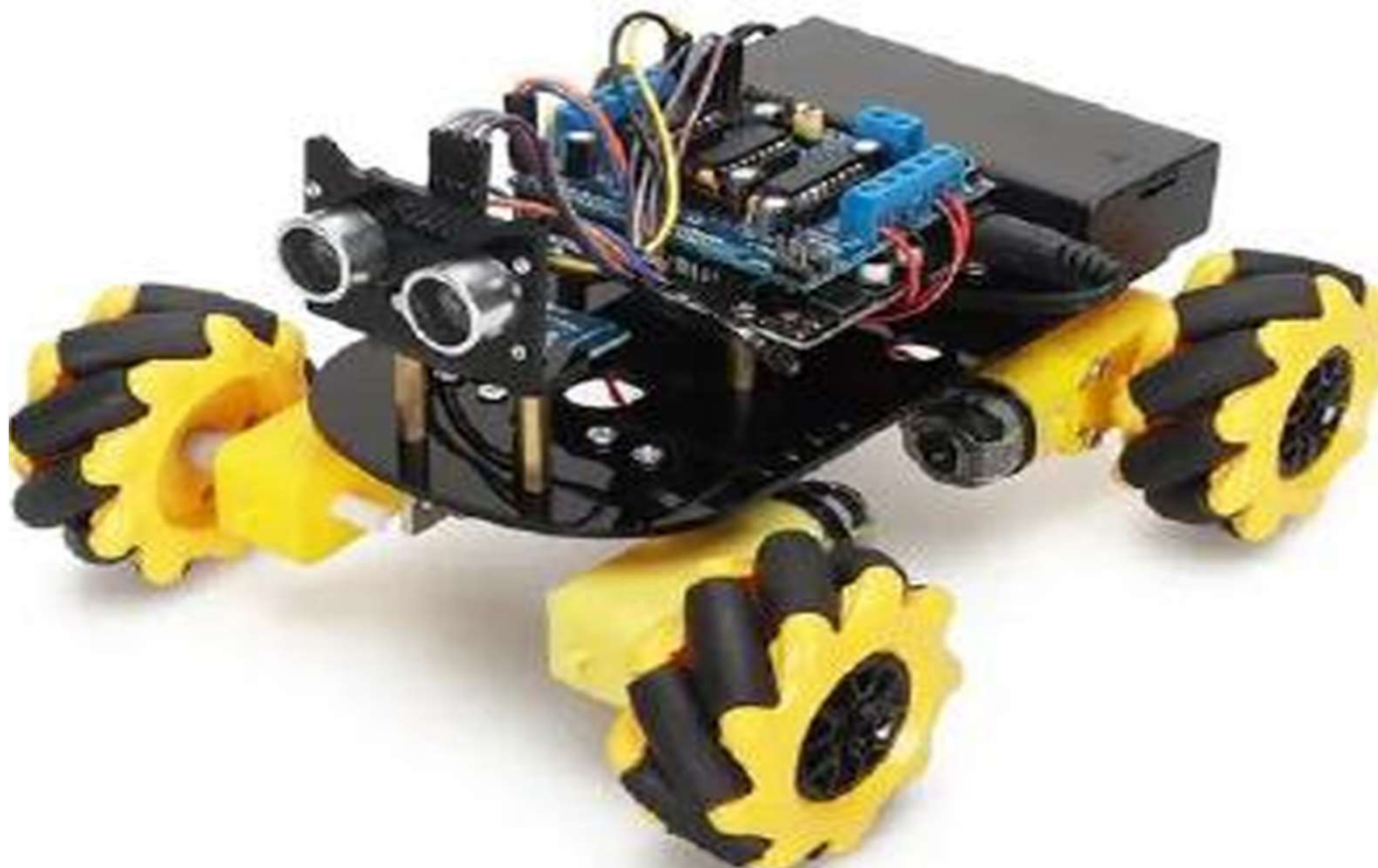




Gp221 - Projet ROVER - Sprint 3

Navigation Robotique - Python



JMR (2025-2026)



Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 - Sommaire

- I. Projet Rover – vue globale**
- II. Objectifs – changement de d’approche (mode projet)**
- III. Déroulement des séances (Module de 20h : 5 x 4h)**
- IV. Rendus / Notation**
- V. Rappels techniques**
 - 1. Modules Python
 - 2. Structure du rover et protocoles de communication
 - 3. Communication avec le Pico / Rover
 - 4. Point rapide sur l’asservissement
- VI. Analyse du code de contrôle de départ**
- VII. Choix du projet**



Gp221 - Projet ROVER - Vue globale

Sprint 1

Concevoir un Rover **démontable** et assembler tous les composants sans problèmes de câblage ni de frottements.

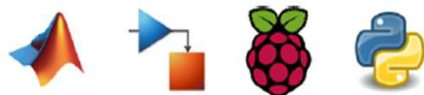
Thèmes : Dimensionnement mécanique et électrique, CAO, Impression 3D, Découpe laser



Sprint 2

Commander les déplacements du Rover avec un microcontrôleur et implémenter un correcteur de trajectoire.

Thèmes : Asservissement de vitesse, Automatique, Python, Modélisation et simulation avec Matlab et Simulink



Sprint 3

Naviguer de façon autonome et détecter des obstacles pour adapter la trajectoire du Rover.

Thèmes : Détection ultrason, Traitement des données



Sprint 4

Piloter le Rover à distance, créer une interface de contrôle et supervision.

Thèmes : Contrôle à distance via Wi-Fi, API Python



Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 - Objectifs – changement de d'approche (mode projet)



Etape 1 → Ecrire les commandes de **haut niveau** pour rendre le **ROVER autonome**



- **Bas niveau** : travail sur les **protocoles de communication** ou l'électronique (**sprint 2**)
- **Haut niveau** : fonctions simples pour des **mouvements élémentaires (avancer, tourner,...)**
- **Autonome** : pas de pilotage en temps réel du rover (il fait sa mission seul)

Etape 2 → Travailler sur un **projet** à définir en équipe en utilisant les **commandes**



- **Exemple** : éviter un obstacle, sortir d'un labyrinthe, comparaison théorie/réalité, ...
- **Faisabilité à vérifier**

Etape 3 → **Passage en mode projet**



- **Sprint 2**: mode TP (liste d'instructions à effectuer)
- **Sprint 3** : mode projet



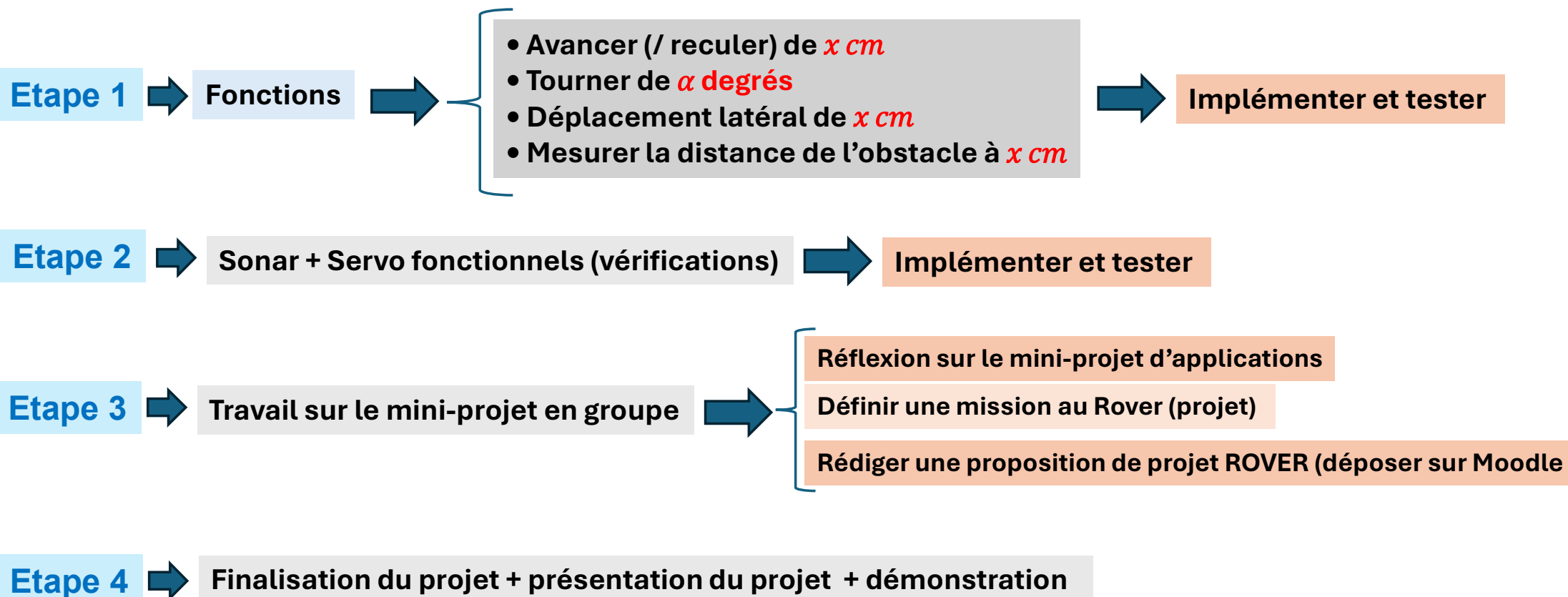
- **Répartition du travail** entre membres de l'équipe
- Définition de **sous objectifs**
- Travail en autonomie
- Soutenance + rapport final + logicielle à la fin

JMR (2025-2026)



Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 – Fonctions à implémenter + Définition du projet par groupe



Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 – Rendus / Notes

**Présentation orale en groupe
à la dernière séance**



- **Démonstration en direct ou en vidéo serait un plus**
- **Durée : 15 min / groupe**
- **Présentation du projet en PWP** en présentant la **structure du projet** + fonctions implémentes

Rendus



- **Les slides et le code (+ fichiers) doivent être rendus sur Moodle**
- **La motivation et les initiatives comptent dans la note !**

JMR (2025-2026)



Gp221 - Projet ROVER - Navigation Robotique

Sprint 2 ➡ Sprint 3

Sprint 2 (TP)

- Utilisation de Matlab/Simulink pour approcher simplement l'asservissement (PID)
- Manipulation du module **Python** qui contrôle les moteurs et qui récupère le nombre de tours que font les roues pour s'assurer que la roue tourne à la bonne vitesse
- Si elle ne tourne pas à la bonne vitesse on applique une **correction** → **asservissement**

Sprint 3 (TP)

- On ne fait que du **Python** ! On utilise un asservissement **PI** en Python!
- On part tous avec le même **module de contrôle** disponible sur Moodle) : **IPSA_Rover_lib.py**
- On écrit un « **main.py** » qui importe le module « **IPSA_Rover_lib.py** » fourni
- On n'utilise qu'une seule fonction « **control_motor_speed** » de ce script

JMR (2025-2026)



Gp221 - Projet ROVER - Navigation Robotique

Rappels Python : Modules

- Bibliothèques, libraries, modules : **globalement identique**
- Lecture d'un fichier avec Python : **OPEN**
- Lecture/exécution d'un fichier Python : **IMPORT**

Scripts Python qui finissent par une partie : **if `__name__` == "`__main__`" :**



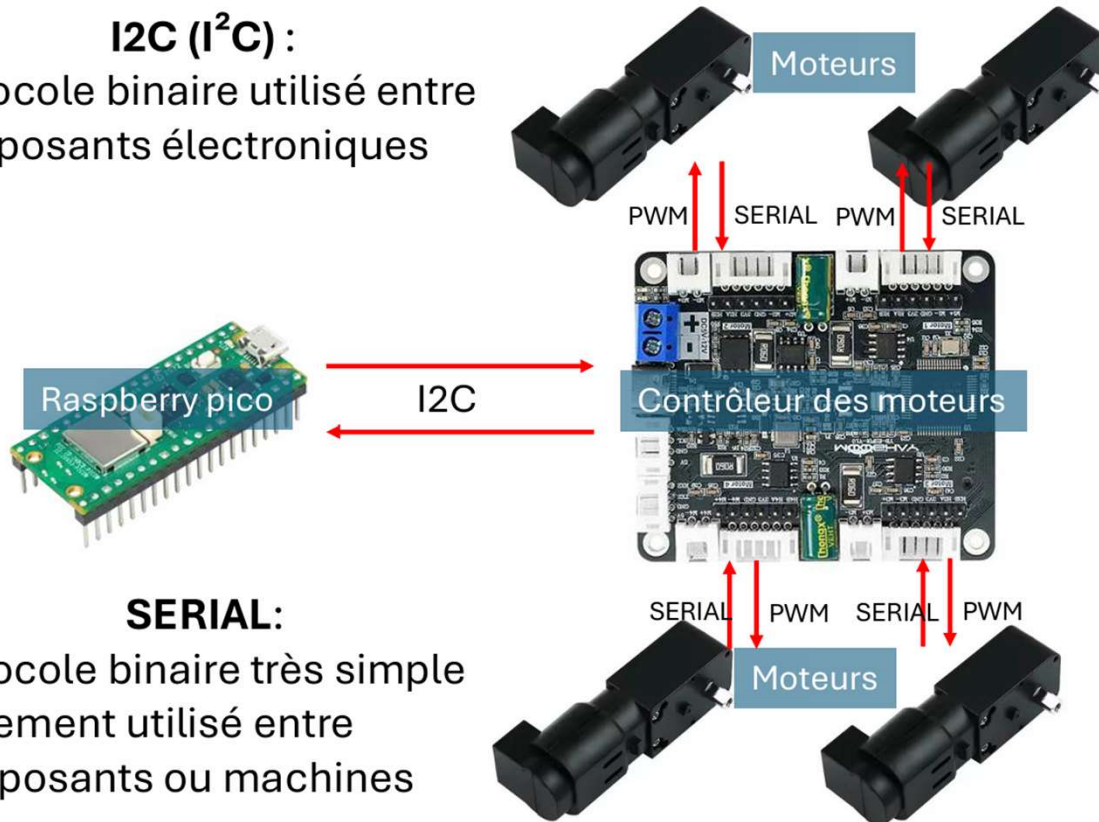
Pour avoir une zone de code qui s'exécute quand c'est ce script qui est **exécuté** mais qui ne s'exécute pas si le script est **importé** !

Gp221 - Projet ROVER - Navigation Robotique

Structure du Rover et protocoles - Partie motorisation

I2C (I²C) :

Protocole binaire utilisé entre composants électroniques



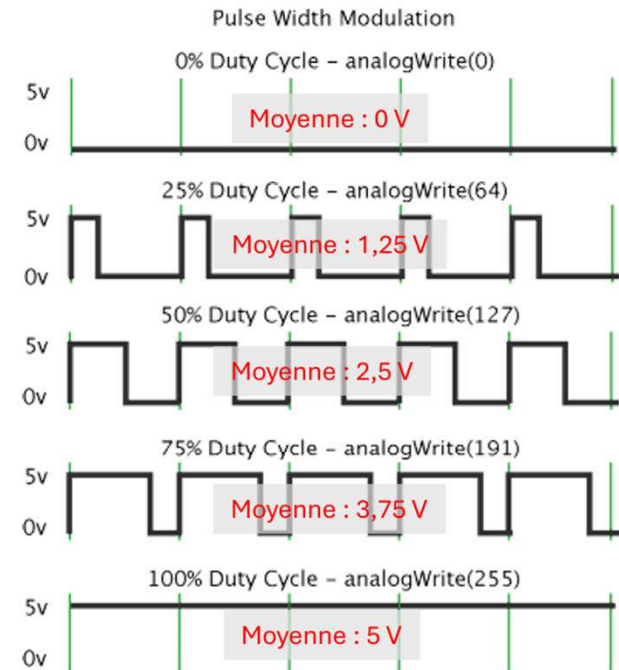
SERIAL:

Protocole binaire très simple également utilisé entre composants ou machines

Note : dans ce *sprint* vous ne manipulerez pas directement ces protocoles !

PWM (Pulse Width Modulation) :

Produire un signal analogique (valeurs continues de tension) avec un signal numérique (que des 0V ou 5V)



Duty Cycle (EN) : Rapport Cyclique (FR)

JMR (2025-2026)

Gp221 - Projet ROVER - Navigation Robotique

Les ticks de encodeur

Encodeur

C'est le capteur qui est inclus dans chaque moteur est qui sert à compter le nombre de tour qu'à fait le moteur

Un tick

C'est une impulsion envoyée par le contrôleur des moteurs au **Raspberry Pico**

Mesure de position

Servo-moteur

Moteur + Réducteur

Augmenter le couple et diminuer la vitesse de rotation de l'axe

$N \approx 2340 \text{ ticks/tours}$

Lecture des ticks

Asservissement des moteurs

L'asservissement des moteurs est réalisé directement avec le contrôleur PI qui lit les ticks (embarqué)

Calcul de la distance parcourue par le Rover

C'est utile pour vérifier la distance parcourue par le Rover

Gp221 - Projet ROVER - Navigation Robotique

Paramètres du ROVER

Paramètres connus (issus de IPSA_Rover_Lib.py)

Diamètre roue → $D = 60 \text{ mm} = 0.06 \text{ m}$

Rayon → $R = 0.03 \text{ m}$

Périmètre de la roue → $C = \pi D$

Ticks par révolution → $N_{ticks} = 2340$

Conversion distance → angle de roue
(Pour une distance linéaire d)

Angle de rotation de la roue → $\theta = \frac{d}{R}$ (en radians)

Nombre de révolutions → $N_{rev} = \frac{d}{\pi D}$

Nombre de ticks attendus → $N_{ticks} = N_{rev} \times 2340$

Gp221 - Projet ROVER - Navigation Robotique

Commander un Servo-moteur



Servo-moteur → C'est un moteur qui maintient un angle précis en fonction du signal qu'il reçoit. → $0^\circ \leftrightarrow 180^\circ$

Signal de Commande → PWM à 50 Hz → revient à 5V toutes les 20 ms

Contrôle des angles →

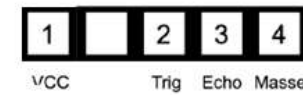
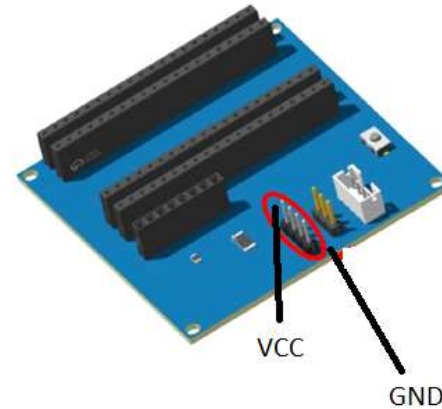
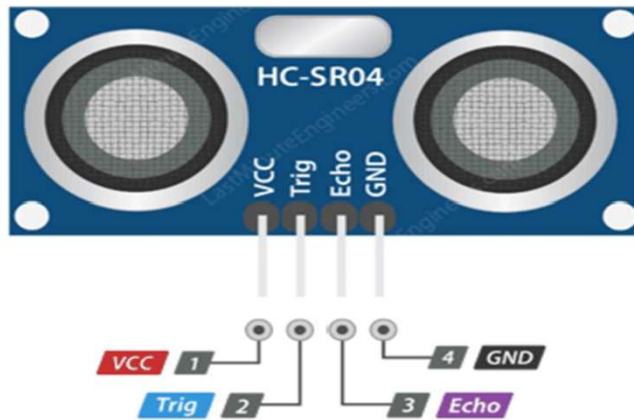
0°	→	5% du rapport cyclique	→	$1000 \mu s$ (1 ms)
180°	→	10 % du rapport cyclique	→	$2000 \mu s$ (2 ms)

Exemple → Si on veut envoyer le moteur à 90° d'angle on envoi un créneau (la commande) de $1500 \mu s$

Note : La plupart des microcontrôleurs (comme le Pico) peuvent facilement générer du signal PWM

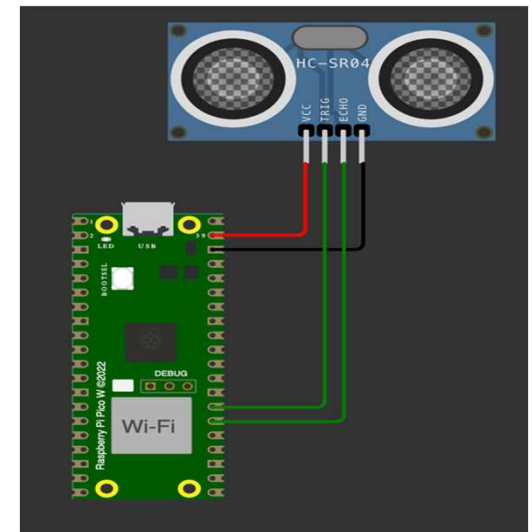
Gp221 - Projet ROVER - Navigation Robotique

Mesurer les distances avec un Sonar - Configuration du capteur à ultrasons



- 1 → Vcc = 5V
- 2 → Trig (Trigger) (Pin GP21) → signal de déclenchement de l'onde incidente (envoyé par le pico)
- 3 → Echo (Pin GP20) → signal déclenché par le retour de l'écho (lue par le pico)
- 4 → GND : Masse

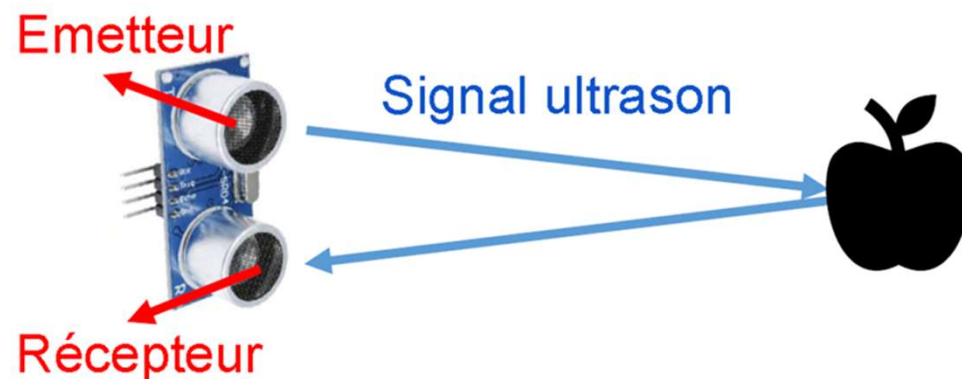
JMR (2025-2026)





Gp221 - Projet ROVER - Navigation Robotique

Mesurer les distances avec un Sonar - Capteur à ultrasons



Vitesse du signal = vitesse du son $c = 340 \text{ m/s}$

d est la distance entre le capteur et l'obstacle

Δt est le temps que le signal met pour parcourir la distance $2.d$ (un aller-retour)

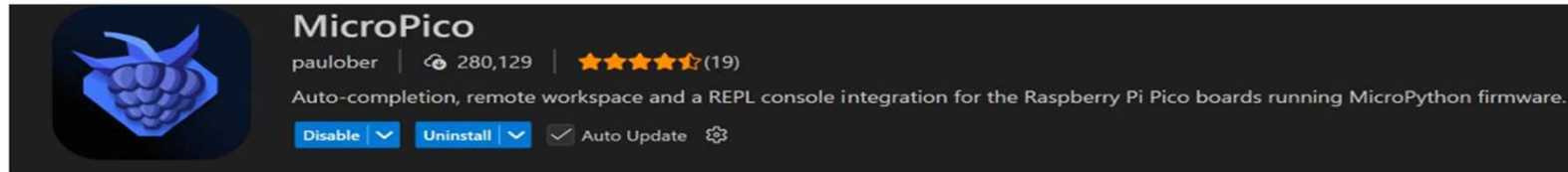
$$c = \frac{2d}{\Delta t} \Rightarrow d = \frac{c \cdot \Delta t}{2}$$

JMR (2025-2026)

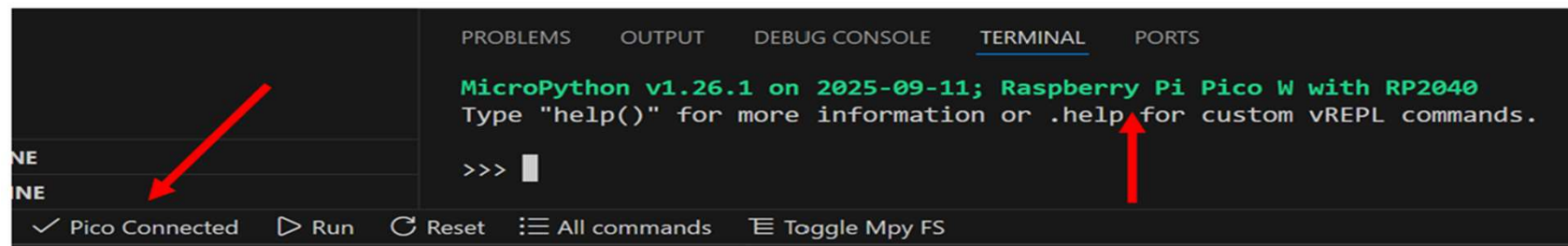
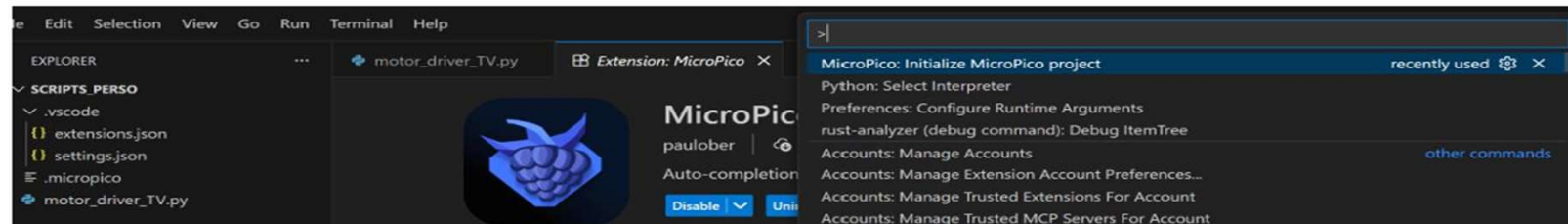
Interface IDE – VSCode

Visual Studio Code (Microsoft)

- Installer l'extension : MicroPico (paulober)

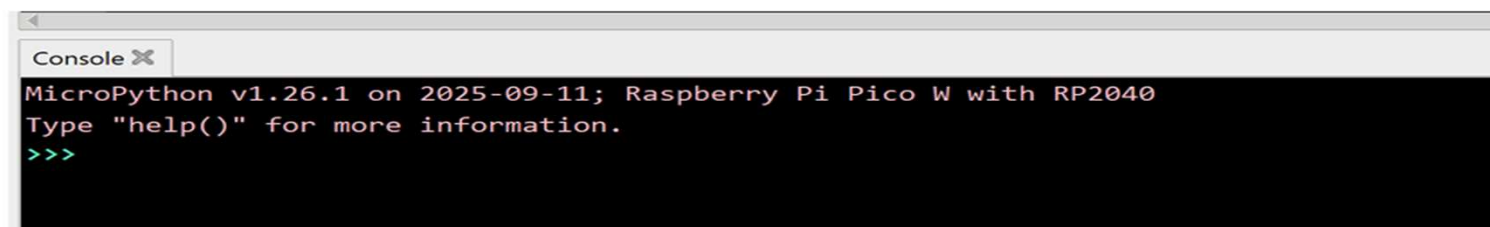
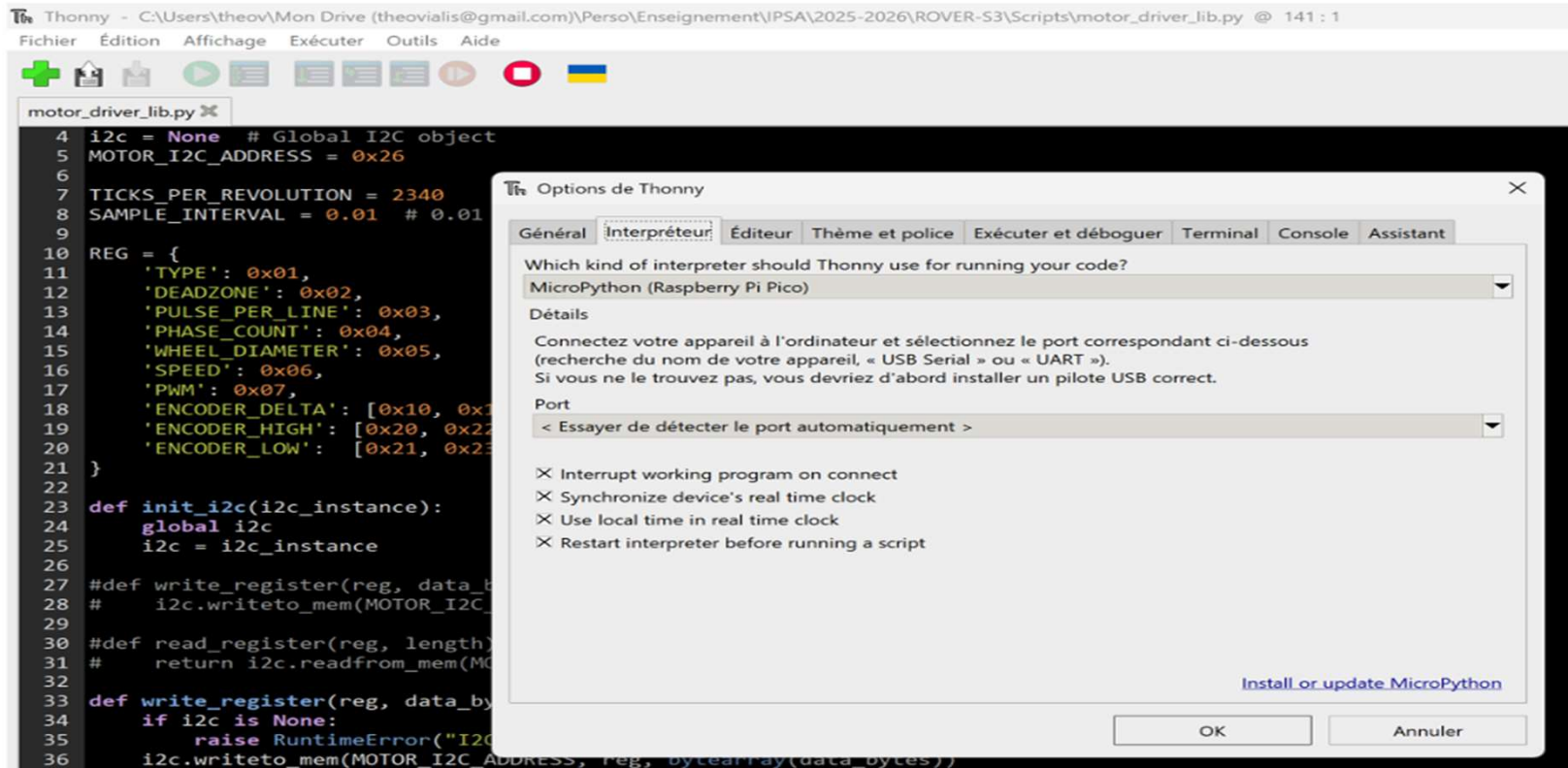


- Initialiser un projet Pico en faisant :
 - Ctrl + MAJ + P → Initialize MicroPico projet



JMR (2025-2026)

Interface IDE – Thonny



JMR (2025-2026)

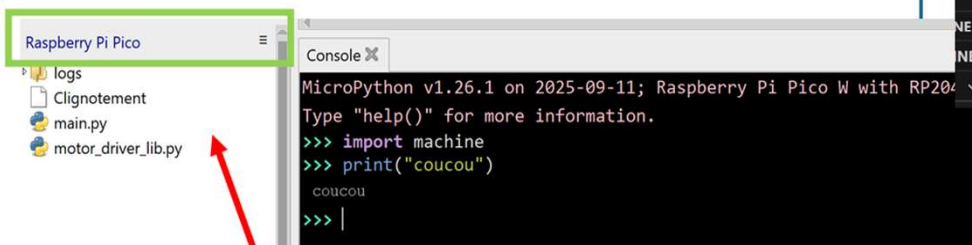


Interface IDE – VSCode

Exécution du code sur le Pico

Exécution automatique

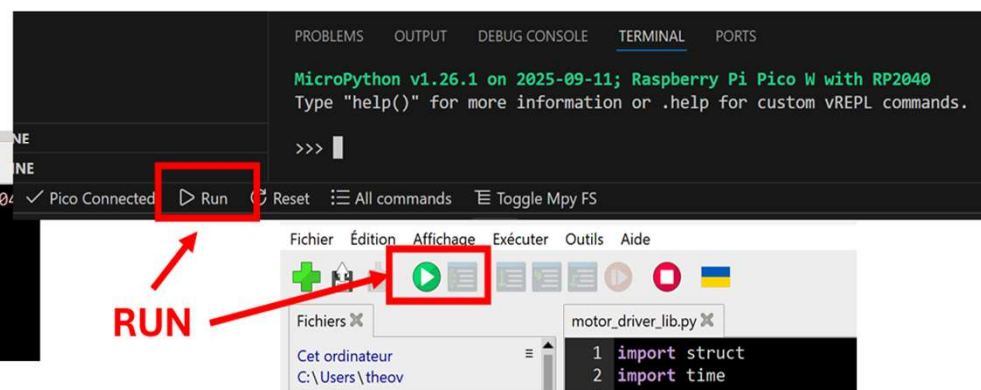
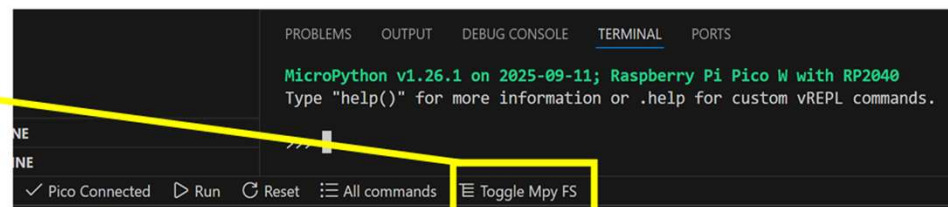
- Un script appelé « main.py » doit être enregistré dans la mémoire interne du Pico (voir les images)



- Les bibliothèques doivent être dans la mémoire du Pico (et pas sur le PC)

Exécution en direct via REPL

- REPL (Read-Eval-Print-Loop) : permet d'exécuter le python du Pico directement sur le PC (le calcul est fait sur le Pico pas sur l'ordinateur)





Automatisme & Asservissement

Automatisme → Faire fonctionner un engin automatiquement → Evènement (E/S)

Asservissement → Algorithme de correction - contrôleur → 2 modes de fonctionnement

Boucle ouverte

Boucle fermée

Boucle ouverte
(Exemple)

Le système exécute une
commande prévue à l'avance

Allumer un moteur pendant 2 s, sans vérifier la position
(système de commande simple)

Boucle fermée
(Asservissement)

Le système exécute une
commande prévue à l'avance

Le système mesure un état, compare à
une consigne et corrige automatiquement

$$e(\text{erreur}) = \text{consigne} - \text{mesure}$$

Exemple: Stabiliser une altitude, garder une orientation, viser une position.



Gp221 - Projet ROVER - Navigation Robotique

Types de Asservissement

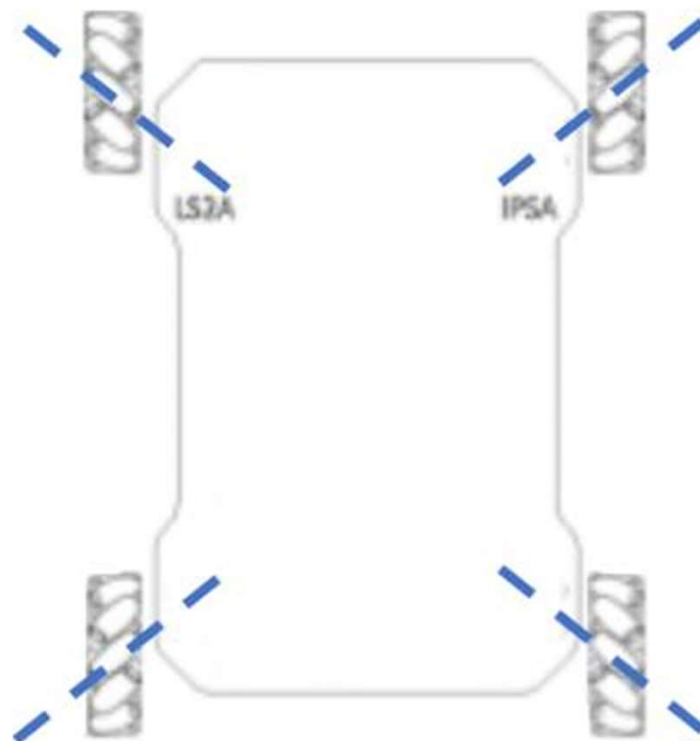
Type	Expression	Effet principal	Complexité
Bang-Bang	commande max/min selon le signe de l'erreur	Réactif mais peu précis	Très simple
P (proportionnel)	$u = K_p e$	Ramène vers la consigne, mais peut osciller	Simple
PD (prop. + dérivé)	$u = K_p e + K_d \dot{e}$	Amortit les oscillations	Moyen
PID (prop. + dérivé + intégral)	$u = K_p e + K_i \int e \, dt + K_d \dot{e}$	Précis et stable	Avancé



Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 - Instructions

Orientation des roues



JMR (2025-2026)

Sprint 3 - Instructions

Motorisation



➡ Créer un nouveau fichier (qui doit s'appeler « **main.py** » si exécution automatique)

➡ Importation des modules ➡

```
from IPSA_Rover_Lib import IpsaRoverLib  
import time
```

➡ Création de l'objet de base ➡

```
driver = IpsaRoverLib()
```

Méthode (fonction) de déplacement ➡ `control_motor_speed(vit_m1, vit_m2, vit_m3, vit_m4)`

➡ **vit_m1** : vitesse moteur 1 (valeurs entre -1000 et +1000)

➡ Valeurs négatives pour avancer (conception du rover)

Exemple ➡

```
time.sleep(5)  
driver.control_motor_speed(-200, -200, -200, -200)  
time.sleep(3)  
driver.control_motor_speed(0, 0, 0, 0)
```

➡ Après 5s, avance à 200 (20% vit max)
pendant 3 sec et arrêt

Attention : fonction non bloquante !

Sprint 3 - Instructions

Motorisation



Vérification de la rotation ➡ `[vit_1, vit_2, vit_3, vit_4]= read_encoder_deltas()`

➡ Donne le nombre de ticks pour 10 ms

```
ticks = driver.read_encoder_deltas()    # compte combien de ticks en 10 mS
print(f"Nombre de ticks : {ticks}")     # pour chaque moteur
RPM = driver.calculate_rpm(ticks[0])    # calcul du rpm pour moteur 1
print(f"RPM : {RPM}")
```

RPM = calculate_rpm(nbr_ticks)

- RPM : tours / minute
- Fait simplement le calcul (avec 2340 ticks/tour et 10 ms)

Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 - Instructions

Servo et Sonar



Rotation du servo-moteur → `set_servo_pulse_us(pulse_us)`

→ Ordonne au Servo d'avoir l'angle correspondant à la valeur du pulse

- 1000 μ s = 0°
- 2000 μ s = 180° (environ)

Exemple

→ Tourner le servo entre 0° et 180° en 20 pas espacés de 100 ms →

```
# Rotation du servo-moteur (sonar)
# Tourner le servo entre 0° et 180° en 20 pas espacés de 100 ms (.1s)
for us in range(1000,2000,50):
    driver.set_servo_pulse_us(us)
    time.sleep(.1)
```

Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 - Instructions

Servo et Sonar



Mesure de la distance devant le sonar ➡ `temps_ms= read_sonar_echo_time_ms()`

➡ Mesure le temps d'aller retour de l'onde sonore donnée en millisecondes

Exemple ➡ Demande 10 fois le temps d'aller-retour de l'onde ultrasonique au Sonar

➡

```
for mes in range(10):  
    echo_time_ms = driver.read_sonar_echo_time_ms()  
    print(f"Temps de echo de l'onde : {echo_time_ms}")  
    time.sleep(.5)
```

Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 - Activité



Bibliothèque
ROVER_mouvements.py

Cinématique simple par odométrie
(encodeurs moteurs)

```
rover_project/
|
|— IPSA_Rover_Lib.py      # fournie
|— ROVER_mouvements.py   # à implémenter
|— main.py               # test des fonctions
```

Déplacement linéaire

def avance(distance, vitesse)

- Déplacement en ligne droite sur une distance réelle (mètres)
- vitesse : [-1000 ; +1000]

Rotation sur place

def tourne(angle_deg, vitesse)

- Rotation du Rover sur place
- Angle_deg : angle en degrés (+ = gauche, - = droite)

Navigation vers
une cible (x,y)

def navigue(x, y, vitesse, angle_depart)

- Navigation vers une position cible (x, y)
- x, y : coordonnées relatives (mètres)
- angle_depart : orientation initiale du Rover (degrés)

Important : vous allez avoir besoin de savoir de combien de cm le rover bouge pour un tour de roue...

calibration

Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 – Programme Main.py



Dans un fichier main.py



Programme de test ROVER
(fonctions avancés)



```
def avance(distance, vitesse)
```

```
def tourne(angle_deg, vitesse)
```

```
def navigue(x, y, vitesse, angle_depart)
```

Importation des modules



```
✓ import time  
from ROVER_mouvements import avance, tourne, navigue  
  
print("Initialisation du Rover...")  
time.sleep(2)
```

Test déplacement linéaire



```
print("\n=== TEST AVANCE ===")  
avance(distance=0.5, vitesse=200) # 50 cm  
time.sleep(1)
```

Gp221 - Projet ROVER - Navigation Robotique

Sprint 3 – Programme Main.py

Test rotation



```
print("\n=== TEST ROTATION ===")  
tourne(angle_deg=90, vitesse=200) # rotation 90°  
time.sleep(1)
```

Test navigation XY



```
print("\n=== TEST NAVIGATION ===")  
angle = 0  
angle = navigue(  
    x=0.4,  
    y=0.3,  
    vitesse=200,  
    angle_depart=angle  
)  
  
print("Nouvel angle =", angle)
```